

SMA + Stigmergy Hybrid Algorithm

3D Drone Simulation in Gazebo + PX4

Complete Beginner Guide — Step by Step

What Are We Building?

This guide walks you through building a complete 3D drone swarm simulation from scratch. By the end, you will have multiple drones flying inside a Gazebo 3D world, each one guided by your SMA+Stigmergy swarm intelligence algorithm, connected to PX4 flight controllers, and monitored live in QGroundControl.

You do not need prior experience with any of these tools. Every command is explained, every concept is defined, and every step builds directly on the previous one.

The Big Picture — How All the Pieces Fit Together

Think of the system as four layers stacked on top of each other:

Term	What It Means
Your Python Script	The brain. Runs your SMA+Stigmergy algorithm, decides where each drone should go next, and sends those decisions down to PX4.
MAVSDK	The messenger. A Python library that translates your algorithm's position commands into MAVLink messages that PX4 understands.
PX4 Flight Controller	The pilot. Receives position targets from your script, stabilizes the drone, and physically moves it there using its own internal control loops.
Gazebo Simulator	The world. Simulates gravity, physics, collisions, and sensor readings in 3D. Shows you what the drones are doing visually.

QGroundControl sits alongside all of this as a monitor — it connects to PX4 and shows you telemetry (position, battery, mode, health) in real time, but it does not control the drones when your algorithm is running.

NOTE: Everything in this guide runs inside WSL2 on Windows. Gazebo, PX4, and your Python script all run inside the WSL Linux environment. QGroundControl runs natively on Windows and connects to PX4 over the network bridge you already set up.

STEP 1: Understand the Key Terms

Before touching any code, make sure you understand what each piece of software actually does. Confusion here causes most beginner mistakes.

Term	What It Means
WSL2	Windows Subsystem for Linux 2. A full Linux environment running inside Windows. All your simulation tools (Gazebo, PX4, Python) run here.
Gazebo Harmonic	A 3D robot/drone simulator. It renders the physical world (terrain, gravity, light), simulates physics (collisions, forces), and runs the drone's sensors (GPS, IMU, barometer) as virtual hardware.
PX4 SITL	Software In The Loop. PX4's flight controller code running as a software process instead of on real hardware. It behaves exactly like a real PX4 autopilot — it reads sensor data from Gazebo, runs its control loops, and outputs motor commands back to Gazebo.
QGroundControl (QGC)	A ground control station app. Connects to PX4 over MAVLink (UDP port 14550) and shows you the drone's GPS position on a map, battery level, arming state, and flight mode. You used this to confirm connectivity in your earlier sessions.
MAVSDK	A Python library for controlling PX4. It speaks the MAVLink protocol so your Python script can arm drones, start offboard mode, and send position setpoints without manually building MAVLink packets.
Offboard Mode	A PX4 flight mode where the flight controller accepts position/velocity/attitude setpoints from an external computer (your Python script) rather than from a human pilot with a joystick. This is the mode your algorithm uses to control the drones.
NED Coordinates	North-East-Down. The coordinate system PX4 and MAVSDK use for positions. North is positive X, East is positive Y, and Down is positive Z (so upward altitude = negative Z). Example: position (10, 5, -8) means 10m North, 5m East, 8m above the takeoff point.
MAVLink	A lightweight messaging protocol used by drones. PX4 speaks MAVLink to communicate with QGC and MAVSDK. You do not need to work with MAVLink directly — MAVSDK handles it for you.
SMA	Slime Mould Algorithm. A swarm optimization algorithm inspired by the foraging behavior of <i>Physarum polycephalum</i> (slime mould). Each drone is an agent that updates its position based on its fitness rank relative to others, balancing

Term	What It Means
	exploration (searching wide) and exploitation (refining near good areas).
Stigmergy	Indirect coordination via a shared environment. Inspired by ant colonies leaving pheromone trails. Your drones deposit a digital pheromone into a shared grid based on how good their current position is. Other drones sense the pheromone gradient and are biased toward moving in directions where pheromone is strong, even if they cannot communicate directly.
Jamming Zone	A region of space where a drone loses access to the swarm's global best position (<code>x_best</code>). In a real-world scenario this would be caused by RF jamming blocking radio communication. In your simulation it is defined as a Python coordinate check: if a drone's position falls inside the zone, it falls back to using its own current position as the reference instead of <code>x_best</code> .
Pheromone Field	A 2D (or 3D) grid overlaid on the world that stores accumulated pheromone values. Each cell decays over time (evaporation), spreads to neighbors (diffusion), and gets topped up by passing drones (deposition). Drones follow the gradient of this field as an additional movement bias.
Fitness Function	A mathematical function $F(\text{position})$ that scores how good a drone's current position is. Lower score = better. Typically: distance to target + penalty if too close to an obstacle. The algorithm always tries to minimize this.

STEP 2: Verify Your Environment Is Ready

Before writing any algorithm code, confirm that every tool from your previous sessions is still working. Run these checks in your WSL terminal every time you start a new session.

2.1 Open a Fresh WSL Terminal

In Windows, open Windows Terminal and select your Ubuntu tab, or run:

```
wsl
```

2.2 Check Gazebo Is Still Accessible

```
gz sim --version
```

Expected output: Gazebo Sim, version 8.10.0 (or similar). If this fails, re-run the `pixi expose` command from your earlier session:

```
pixi global expose add gz=gz --environment gz-harmonic
```

2.3 Check Python and MAVSDK

```
python3 -c "from mavsdk import System; print('MAVSDK OK')"
```

Expected output: MAVSDK OK. If this fails:

```
pip install mavsdk --break-system-packages
```

2.4 Check Your Project Directory

```
ls ~/robotics_sim/PX4-Autopilot/Tools/simulation/gz/models/x500/model.sdf
```

This file must exist and be unmodified. If it was corrupted again, restore it:

```
cd ~/robotics_sim/PX4-Autopilot/Tools/simulation/gz
git checkout -- models/x500/model.sdf
```

2.5 Launch the Simulation

Use the windy world for your algorithm testing. It is an empty open space with no trees or obstacles to crash into accidentally, and it runs at a high real-time factor (90%+) because it has no complex meshes to render.

```
cd ~/robotics_sim/PX4-Autopilot
PX4_SIM_HOST_ADDR=127.0.0.1 make px4_sitl gz_x500_windy
```

Wait until you see all of these lines in the terminal output before proceeding:

```
INFO [gz_bridge] world: windy, model: x500_0
INFO [commander] Ready for takeoff!
INFO [mavlink] partner IP: 127.0.0.1
```

TIP: If the Gazebo window appears black or does not open, the simulation is still running correctly in the background. The 3D view is cosmetic and the algorithm will still work without it.

2.6 Connect QGroundControl

Open QGroundControl on Windows. Click Disconnected at the top, then select WSL Sim. The header should change to show the vehicle is connected and Ready for takeoff. You should see the drone icon on the Baylands map (its GPS home is set to the baylands coordinates even in the windy world).

IMPORTANT: If QGC still shows Disconnected after clicking WSL Sim, your `.wslconfig` mirrored networking mode may have been reset. Re-check: the file `C:\Users\Girijan\.wslconfig` must contain `networkingMode=mirrored` under `[wsl2]`. Run `wsl --shutdown` and reopen WSL if you had to recreate it.

STEP 3: Understand the Algorithm Before Coding It

Read this section carefully. Understanding what each algorithm step does makes the code much easier to write and debug.

3.1 The Algorithm Runs in a Loop

Your SMA+Stigmergy algorithm runs as a repeated loop, once per iteration. In the real 3D simulation, each iteration corresponds to one physical timestep where every drone moves. Here is what happens in each iteration, in order:

1. Evaluate fitness for every drone. Read each drone's current 3D position from PX4's telemetry, then compute $F(\text{position})$ for each one. $F = \text{distance to target} + \text{obstacle penalty}$. Lower is better.
2. Rank drones by fitness. Sort them from best (lowest F) to worst (highest F). The best drone becomes x_{best} , the swarm's current best-known position.
3. Compute adaptive weights. Each drone gets a weight vector W based on its rank. Better-ranked drones get weights above 1.0 (amplifying their movement toward better positions). Worse-ranked drones get weights below 1.0 (dampening their movement so they do not overshoot).
4. Compute oscillation parameters a and b . These two numbers shrink linearly from their starting values toward zero as the iteration count approaches max_iter . Early iterations have large a and b values, encouraging wide exploration. Later iterations have small values, encouraging fine convergence near the target.
5. Update each drone's position. For each drone, one of three things happens: (a) with small probability $z=0.03$, the drone teleports to a random location (exploration). (b) with probability $p=\tanh(|f_i - f_{\text{best}}|)$, the drone moves toward x_{best} modulated by two randomly chosen other drones (approach behavior). (c) otherwise, the drone contracts toward its own current position (refinement behavior). If the drone is inside a jammed zone, x_{best} is replaced by the drone's own current position in step (b).
6. Apply stigmergy bias. Regardless of which movement type was chosen, read the pheromone field gradient at the drone's current position and nudge the new position slightly in the direction of increasing pheromone strength. This step works even when jammed.
7. Cap step size. If the computed move is larger than max_step (5% of world size per iteration), scale it down to max_step . This simulates realistic drone speed limits.
8. Only accept the new position if its fitness is better than the old position's fitness. If the new Boundary clipping and greedy selection. Clip the new position to stay within the world bounds. position is worse, keep the old one.
9. Update pheromone field. Each drone deposits pheromone proportional to its fitness quality into the grid cell at its new position. Then the entire field decays slightly (evaporation) and spreads to neighboring cells (diffusion via Gaussian blur).

3.2 Why This Works for Jamming Resilience

The key insight of your algorithm is that the pheromone field is a local physical signal, not a radio communication. A drone inside a jammed zone cannot hear x_{best} from the rest of the swarm, but it CAN still sense the pheromone gradient at its physical location, because pheromone is treated like a chemical in the environment rather than a transmitted message. This means jammed drones degrade gracefully (following trails left by non-jammed drones) instead of failing completely.

STEP 4: Build the Project File Structure

Organize your code into a clean directory structure before writing anything. This makes debugging much easier later.

```
# Run these commands in your WSL terminal
cd ~/robotics_sim
mkdir -p swarm_project/algorithm
mkdir -p swarm_project/drone_control
mkdir -p swarm_project/config
mkdir -p swarm_project/logs
touch swarm_project/algorithm/__init__.py
touch swarm_project/drone_control/__init__.py
ls swarm_project/
```

Your project structure will look like this:

```
swarm_project/
  algorithm/
    __init__.py
    sma_core.py      <- SMA weight/oscillation/position update logic
    pheromone.py     <- Pheromone field (deposit/evaporate/diffuse)
    fitness.py       <- Fitness function and jamming zone check
  drone_control/
    __init__.py
    connection.py    <- MAVSDK connect/arm/takeoff helpers
    telemetry.py     <- Read drone positions from PX4
    setpoint.py      <- Send position commands to PX4
  config/
    settings.py      <- All hyperparameters in one place
  main.py           <- Entry point: ties everything together
  logs/            <- Flight data saved here
```

STEP 5: Write the Configuration File

Put all hyperparameters in one file so you can tune the algorithm without hunting through the code. Create this file:

```
nano ~/robotics_sim/swarm_project/config/settings.py
```

Paste the following content:

```
# =====
# settings.py - All tunable parameters in one place
# =====

import numpy as np

# --- World / Search Space (NED coordinates, meters) -----
# This defines the 3D box your drones are allowed to fly inside.
# North/East bounds: how far from home position in each direction
# Altitude bounds: how high (negative Z = up in NED)
WORLD_BOUNDS = {
    'north': (-50.0, 50.0), # meters North of home
    'east':  (-50.0, 50.0), # meters East of home
    'alt':   (-25.0, -2.0), # altitude: -2m min, -25m max (above ground)
}

# --- Target Position (NED, meters from home) -----
TARGET = np.array([40.0, 40.0, -15.0]) # 40m North, 40m East, 15m high

# --- Obstacle (sphere in 3D space) -----
OBSTACLE_CENTER = np.array([20.0, 20.0, -12.0])
```

```

OBSTACLE_RADIUS = 8.0      # meters
OBSTACLE_PENALTY = 100.0   # added to fitness when inside obstacle radius

# --- Jammed Zone (rectangular box in NED space) -----
JAMMED_ZONE = {
    'north_min': 5.0, 'north_max': 30.0,
    'east_min': 5.0, 'east_max': 30.0,
    'alt_min': -20.0, 'alt_max': -5.0,
}

# --- Swarm Parameters -----
N_AGENTS = 3               # number of drones (match your PX4 instances)
MAX_ITER = 100             # how many algorithm iterations to run
MAX_STEP = 3.0            # max meters a drone can move per iteration
Z_RANDOM = 0.03           # probability of pure random exploration per step

# --- Pheromone Field -----
GRID_SIZE = 30            # resolution of pheromone grid per axis (3D = 30^3)
EVAP_RATE = 0.05          # fraction of pheromone lost per iteration
PHEROMONE_WEIGHT = 0.2    # how strongly pheromone biases movement
GAUSSIAN_SIGMA = 1.0      # pheromone diffusion spread (grid cells)

# --- MAVSDK Connection Ports -----
# Each drone/PX4 instance needs a unique port.
# Instance 0 uses 14540, instance 1 uses 14541, etc.
DRONE_PORTS = [14540, 14541, 14542] # one per drone

# --- Timing -----
ITERATION_DELAY = 2.0     # seconds to wait between iterations
                        # give drones time to physically reach setpoints

```

NOTE: Start with `N_AGENTS=1` for your first test run. Add more drones only after confirming single-drone control works correctly.

STEP 6: Write the Fitness Function

```

nano ~/robotics_sim/swarm_project/algorithm/fitness.py

# =====
# fitness.py - How we score each drone's position
# =====

import numpy as np
from config.settings import TARGET, OBSTACLE_CENTER, OBSTACLE_RADIUS
from config.settings import OBSTACLE_PENALTY, JAMMED_ZONE

def fitness(positions):
    """
    Compute fitness F(x) for each drone position.
    positions: numpy array of shape (N, 3) - N drones, each with (north, east, alt)
    Returns: numpy array of shape (N,) - one fitness score per drone
    Lower score = better (closer to target, away from obstacle).
    """
    # Part 1: Euclidean distance from each drone to the target
    dist_to_target = np.linalg.norm(positions - TARGET, axis=1)

    # Part 2: Obstacle penalty
    # If a drone is closer than OBSTACLE_RADIUS to the obstacle center,
    # add a penalty proportional to how deep inside the obstacle it is.
    dist_to_obstacle = np.linalg.norm(positions - OBSTACLE_CENTER, axis=1)
    penalty = np.where(

```

```

        dist_to_obstacle < OBSTACLE_RADIUS,
        OBSTACLE_PENALTY * (1.0 - dist_to_obstacle / OBSTACLE_RADIUS),
        0.0
    )

    return dist_to_target + penalty

def is_jammed(positions):
    """
    Check which drones are inside the jammed zone.
    positions: numpy array of shape (N, 3)
    Returns: boolean array of shape (N,) — True means jammed
    """
    n = positions[:, 0] # north values
    e = positions[:, 1] # east values
    a = positions[:, 2] # altitude values

    return (
        (n >= JAMMED_ZONE['north_min']) & (n <= JAMMED_ZONE['north_max']) &
        (e >= JAMMED_ZONE['east_min']) & (e <= JAMMED_ZONE['east_max']) &
        (a >= JAMMED_ZONE['alt_min']) & (a <= JAMMED_ZONE['alt_max'])
    )

```

STEP 7: Write the Pheromone Field

```

nano ~/robotics_sim/swarm_project/algorithm/pheromone.py

# =====
# pheromone.py - 3D pheromone grid (deposit/evaporate/diffuse)
# =====

import numpy as np
from scipy.ndimage import gaussian_filter
from config.settings import WORLD_BOUNDS, GRID_SIZE, EVAP_RATE, GAUSSIAN_SIGMA

class PheromoneField3D:
    """
    A 3D grid of pheromone values overlaid on the world.
    Grid shape: (GRID_SIZE, GRID_SIZE, GRID_SIZE) covering
    (north, east, altitude) bounds from settings.
    """

    def __init__(self):
        self.grid = np.zeros((GRID_SIZE, GRID_SIZE, GRID_SIZE))
        # Compute cell sizes for each axis
        self.n_range = WORLD_BOUNDS['north']
        self.e_range = WORLD_BOUNDS['east']
        self.a_range = WORLD_BOUNDS['alt']

    def _to_cell(self, positions):
        """Convert NED world positions to grid cell indices."""
        n_idx = ((positions[:, 0] - self.n_range[0]) /
                  (self.n_range[1] - self.n_range[0]) * (GRID_SIZE - 1)).astype(int)
        e_idx = ((positions[:, 1] - self.e_range[0]) /
                  (self.e_range[1] - self.e_range[0]) * (GRID_SIZE - 1)).astype(int)
        a_idx = ((positions[:, 2] - self.a_range[0]) /
                  (self.a_range[1] - self.a_range[0]) * (GRID_SIZE - 1)).astype(int)
        # Clamp to valid grid bounds
        n_idx = np.clip(n_idx, 0, GRID_SIZE - 1)
        e_idx = np.clip(e_idx, 0, GRID_SIZE - 1)

```

```

        a_idx = np.clip(a_idx, 0, GRID_SIZE - 1)
        return n_idx, e_idx, a_idx

    def deposit(self, positions, qualities):
        """
        Each drone deposits pheromone at its grid cell.
        qualities: array of shape (N,), values in [0,1].
        Higher quality = drone is in a good location = more pheromone deposited.
        """
        n_idx, e_idx, a_idx = self._to_cell(positions)
        for i in range(len(positions)):
            self.grid[n_idx[i], e_idx[i], a_idx[i]] += qualities[i]

    def evaporate(self):
        """Multiply entire grid by (1 - evap_rate) - pheromone fades over time."""
        self.grid *= (1.0 - EVAP_RATE)

    def diffuse(self):
        """Spread pheromone to neighboring cells using a Gaussian blur."""
        self.grid = gaussian_filter(self.grid, sigma=GAUSSIAN_SIGMA)

    def gradient_at(self, pos):
        """
        Compute the pheromone gradient at a single world position.
        Returns a 3D vector pointing toward higher pheromone.
        pos: shape (3,) - single drone's (north, east, alt)
        """
        pos_arr = pos.reshape(1, 3)
        ni, ei, ai = [x[0] for x in self._to_cell(pos_arr)]

        # Finite differences: sample neighbors in each axis
        ni_p, ni_m = min(ni+1, GRID_SIZE-1), max(ni-1, 0)
        ei_p, ei_m = min(ei+1, GRID_SIZE-1), max(ei-1, 0)
        ai_p, ai_m = min(ai+1, GRID_SIZE-1), max(ai-1, 0)

        d_north = self.grid[ni_p, ei, ai] - self.grid[ni_m, ei, ai]
        d_east  = self.grid[ni, ei_p, ai] - self.grid[ni, ei_m, ai]
        d_alt   = self.grid[ni, ei, ai_p] - self.grid[ni, ei, ai_m]

        return np.array([d_north, d_east, d_alt])

```

STEP 8: Write the SMA Core Logic

```

nano ~/robotics_sim/swarm_project/algorithm/sma_core.py

# =====
# sma_core.py - SMA weight/oscillation/position update logic
# =====

import numpy as np
from config.settings import (MAX_ITER, MAX_STEP, Z_RANDOM, PHEROMONE_WEIGHT,
                             WORLD_BOUNDS)
from algorithm.fitness import fitness, is_jammed

def clip_to_bounds(positions):
    """Keep all drone positions inside the defined world box."""
    clipped = positions.copy()
    clipped[:, 0] = np.clip(positions[:, 0], *WORLD_BOUNDS['north'])
    clipped[:, 1] = np.clip(positions[:, 1], *WORLD_BOUNDS['east'])
    clipped[:, 2] = np.clip(positions[:, 2], *WORLD_BOUNDS['alt'])
    return clipped

```

```

def compute_weights(sorted_idx, fitness_vals):
    """
    Step 2: compute per-agent adaptive weight vectors.
    Better-ranked agents get weights above 1 (amplifying their moves).
    Worse-ranked agents get weights below 1 (dampening their moves).
    """
    n = len(fitness_vals)
    f_best = fitness_vals[sorted_idx[0]]
    f_worst = fitness_vals[sorted_idx[-1]]
    denom = max(abs(f_best - f_worst), 1e-10)

    weights = np.ones((n, 3)) # 3D weight vectors
    for rank, idx in enumerate(sorted_idx):
        r = np.random.uniform(0, 1, size=3) # random vector
        log_val = np.log10((f_best - fitness_vals[idx]) / denom + 1)
        if rank < n / 2:
            weights[idx] = 1 + r * log_val # amplify for top half
        else:
            weights[idx] = 1 - r * log_val # dampen for bottom half
    return weights

def sma_step(positions, fitness_vals, x_best, pheromone_field, t):
    """
    One full SMA+Stigmergy iteration.
    positions:      numpy array (N, 3) - current drone positions in NED
    fitness_vals:   numpy array (N,) - current fitness scores
    x_best:         numpy array (3,) - best position seen so far
    pheromone_field: PheromoneField3D - shared pheromone grid
    t:              int - current iteration number
    Returns: (new_positions, new_fitness_vals)
    """
    n = len(positions)
    new_positions = positions.copy()

    # Rank agents by fitness (ascending)
    sorted_idx = np.argsort(fitness_vals)
    f_best = fitness_vals[sorted_idx[0]]
    f_worst = fitness_vals[sorted_idx[-1]]

    # Step 2: weights
    weights = compute_weights(sorted_idx, fitness_vals)

    # Step 3: oscillation parameters (shrink over time)
    frac = t / MAX_ITER
    a = float(np.arctanh(np.clip(-frac + 1, -0.9999, 0.9999)))
    b = 1.0 - frac

    # Step 4: per-agent position update
    jammed = is_jammed(positions)

    for i in range(n):

        # (a) Pure random exploration
        if np.random.rand() < Z_RANDOM:
            new_positions[i] = np.array([
                np.random.uniform(*WORLD_BOUNDS['north']),
                np.random.uniform(*WORLD_BOUNDS['east']),
                np.random.uniform(*WORLD_BOUNDS['alt']),
            ])
            continue

```

```

p_i = np.tanh(abs(fitness_vals[i] - f_best))

if np.random.rand() < p_i:
    # (b) Approach behavior
    others = [j for j in range(n) if j != i]
    if len(others) >= 2:
        A, B = np.random.choice(others, size=2, replace=False)
    else:
        A = B = others[0] if others else i

    v_b = np.random.uniform(-a, a, size=3)

    # KEY: jammed drones use self as reference, not x_best
    x_ref = positions[i] if jammed[i] else x_best
    new_positions[i] = x_ref + v_b * (weights[i] * positions[A] - positions[B])

else:
    # (c) Contraction behavior
    v_c = np.random.uniform(-b, b, size=3)
    new_positions[i] = v_c * positions[i]

# Step 5: stigmergy bias (works even when jammed)
grad = pheromone_field.gradient_at(positions[i])
grad_norm = np.linalg.norm(grad)
if grad_norm > 1e-8:
    new_positions[i] += PHEROMONE_WEIGHT * (grad / grad_norm) * MAX_STEP

# Step 6: cap step size
step = new_positions[i] - positions[i]
step_norm = np.linalg.norm(step)
if step_norm > MAX_STEP:
    step = (step / step_norm) * MAX_STEP
new_positions[i] = positions[i] + step

# Step 7: boundary clip + greedy selection
new_positions = clip_to_bounds(new_positions)
new_fitness = fitness(new_positions)
accept = new_fitness < fitness_vals
final_pos = np.where(accept[:, None], new_positions, positions)
final_fit = np.where(accept, new_fitness, fitness_vals)

# Step 8: pheromone update
denom = max(abs(final_fit.max() - final_fit.min()), 1e-10)
qualities = (final_fit.max() - final_fit) / denom
pheromone_field.deposit(final_pos, qualities)
pheromone_field.evaporate()
pheromone_field.diffuse()

return final_pos, final_fit

```

STEP 9: Write the MAVSDK Drone Control Helpers

9.1 Connection and Arming Helper

```

nano ~/robotics_sim/swarm_project/drone_control/connection.py

# =====
# connection.py - Connect, arm, and start offboard mode
# =====

import asyncio

```

```

from mavsdk import System
from mavsdk.offboard import PositionNedYaw

async def connect_drone(port):
    """
    Connect to a PX4 instance via MAVSDK on the given UDP port.
    Returns the connected System object.
    """
    drone = System()
    await drone.connect(system_address=f'udpin://0.0.0.0:{port}')

    print(f'[Port {port}] Waiting for connection...')
    async for state in drone.core.connection_state():
        if state.is_connected:
            print(f'[Port {port}] Connected!')
            break

    return drone

async def arm_and_start_offboard(drone, port):
    """
    Arm the drone and switch to offboard mode.
    Must send an initial setpoint before calling offboard.start().
    """
    print(f'[Port {port}] Waiting for GPS/sensor readiness...')
    async for health in drone.telemetry.health():
        if health.is_global_position_ok and health.is_home_position_ok:
            print(f'[Port {port}] GPS ready.')
            break

    print(f'[Port {port}] Arming...')
    await drone.action.arm()

    # Send initial setpoint - REQUIRED before offboard.start()
    await drone.offboard.set_position_ned(PositionNedYaw(0.0, 0.0, 0.0, 0.0))
    await drone.offboard.start()
    print(f'[Port {port}] Offboard mode started.')

    # Command initial takeoff altitude
    print(f'[Port {port}] Taking off...')
    await drone.offboard.set_position_ned(PositionNedYaw(0.0, 0.0, -10.0, 0.0))
    await asyncio.sleep(8) # wait for drone to reach altitude
    print(f'[Port {port}] At altitude. Ready for algorithm control.')

```

9.2 Telemetry Reader

```

nano ~/robotics_sim/swarm_project/drone_control/telemetry.py

# =====
# telemetry.py - Read NED position from PX4
# =====

import numpy as np

async def get_ned_position(drone):
    """
    Read the drone's current NED position from PX4 telemetry.
    Returns a numpy array: [north_m, east_m, altitude_m]

```

```
Note: PX4 uses Down as positive Z. We flip it so up = positive.
"""
async for pos in drone.telemetry.position_velocity_ned():
    return np.array([
        pos.position.north_m,
        pos.position.east_m,
        -pos.position.down_m,    # flip: up is positive in our system
    ])

```

9.3 Setpoint Sender

```
nano ~/robotics_sim/swarm_project/drone_control/setpoint.py

# =====
# setpoint.py - Send algorithm's output to PX4 as a setpoint
# =====

from mavsdk.offboard import PositionNedYaw

async def send_ned_setpoint(drone, target_pos, yaw_deg=0.0):
    """
    Command a drone to fly to target_pos.
    target_pos: numpy array [north_m, east_m, altitude_m]
    PX4 NED uses Down as positive Z, so we flip altitude back.
    """
    await drone.offboard.set_position_ned(
        PositionNedYaw(
            float(target_pos[0]),    # North
            float(target_pos[1]),    # East
            float(-target_pos[2]),   # Down (flip altitude sign back)
            yaw_deg,                 # Yaw heading in degrees
        )
    )

```

STEP 10: Write the Main Entry Point

```
nano ~/robotics_sim/swarm_project/main.py

# =====
# main.py - Entry point: ties all pieces together
# =====

import asyncio
import numpy as np
import json
from datetime import datetime

from config.settings import (N_AGENTS, MAX_ITER, DRONE_PORTS,
                             ITERATION_DELAY, WORLD_BOUNDS)
from algorithm.fitness import fitness
from algorithm.pheromone import PheromoneField3D
from algorithm.sma_core import sma_step
from drone_control.connection import connect_drone, arm_and_start_offboard
from drone_control.telemetry import get_ned_position
from drone_control.setpoint import send_ned_setpoint

async def main():
    print('=== SMA + Stigmergy 3D Drone Swarm ===')
```

```

print(f'Drones: {N_AGENTS} | Iterations: {MAX_ITER}')
print()

# ---- 1. Connect all drones ----
drones = []
for i in range(N_AGENTS):
    drone = await connect_drone(DRONE_PORTS[i])
    drones.append(drone)

# ---- 2. Arm and take off all drones ----
await asyncio.gather(*[
    arm_and_start_offboard(d, DRONE_PORTS[i])
    for i, d in enumerate(drones)
])

# ---- 3. Read initial positions ----
positions = np.zeros((N_AGENTS, 3))
for i, drone in enumerate(drones):
    positions[i] = await get_ned_position(drone)
print(f'Initial positions read: {positions}')

fitness_vals = fitness(positions)
x_best = positions[np.argmin(fitness_vals)].copy()
pheromone_field = PheromoneField3D()

# ---- 4. Log setup ----
log_data = []
run_id = datetime.now().strftime('%Y%m%d_%H%M%S')

print()
print('=== Starting algorithm loop ===')

# ---- 5. Main algorithm loop ----
for t in range(MAX_ITER):
    print(f'--- Iteration {t+1}/{MAX_ITER} ---')
    print(f'    Best fitness so far: {fitness_vals.min():.3f}')
    print(f'    x_best: {x_best}')

    # Run one SMA+Stigmergy step
    new_positions, fitness_vals = sma_step(
        positions, fitness_vals, x_best, pheromone_field, t
    )

    # Update global best
    best_idx = np.argmin(fitness_vals)
    if fitness_vals[best_idx] < fitness(x_best.reshape(1, 3))[0]:
        x_best = new_positions[best_idx].copy()
        print(f'    New x_best found: {x_best}')

    # Send setpoints to all drones simultaneously
    await asyncio.gather(*[
        send_ned_setpoint(drones[i], new_positions[i])
        for i in range(N_AGENTS)
    ])

    positions = new_positions

    # Log this iteration
    log_data.append({
        'iteration': t + 1,
        'best_fitness': float(fitness_vals.min()),
        'x_best': x_best.tolist(),
        'positions': positions.tolist(),
    })

```

```

        # Wait for drones to physically move before next iteration
        await asyncio.sleep(ITERATION_DELAY)

    # ---- 6. Land all drones ----
    print()
    print('=== Algorithm complete. Landing all drones. ===')
    await asyncio.gather(*[d.action.land() for d in drones])

    # ---- 7. Save log ----
    log_path = f'logs/run_{run_id}.json'
    with open(log_path, 'w') as f:
        json.dump(log_data, f, indent=2)
    print(f'Log saved to {log_path}')
    print(f'Final best fitness: {fitness_vals.min():.3f}')
    print(f'Final best position: {x_best}')

if __name__ == '__main__':
    asyncio.run(main())

```

STEP 11: Install Dependencies

```
pip install mavsdk numpy scipy --break-system-packages
```

Verify all imports work before running:

```

cd ~/robotics_sim/swarm_project
python3 -c "
import numpy, scipy, asyncio
from mavsdk import System
from config.settings import N_AGENTS, TARGET
from algorithm.fitness import fitness
from algorithm.pheromone import PheromoneField3D
from algorithm.sma_core import sma_step
print('All imports OK')
"

```

Expected output: All imports OK

STEP 12: Run the Single-Drone Test First

IMPORTANT: Always test with N_AGENTS=1 first. Make sure settings.py has N_AGENTS = 1 and DRONE_PORTS = [14540] before your first run.

12.1 Make Sure PX4 + Gazebo Is Running

In Terminal 1 (WSL), if not already running:

```

cd ~/robotics_sim/PX4-Autopilot
PX4_SIM_HOST_ADDR=127.0.0.1 make px4_sitl gz_x500_windy

```

Wait for: INFO [commander] Ready for takeoff!

12.2 Run the Algorithm Script

Open a NEW WSL terminal tab (do not use the PX4 terminal), then:

```
cd ~/robotics_sim/swarm_project
python3 main.py
```

12.3 What You Should See

In your terminal:

```
=== SMA + Stigmergy 3D Drone Swarm ===
Drones: 1 | Iterations: 100
[Port 14540] Waiting for connection...
[Port 14540] Connected!
[Port 14540] GPS ready.
[Port 14540] Arming...
[Port 14540] Offboard mode started.
[Port 14540] Taking off...
[Port 14540] At altitude. Ready for algorithm control.
Initial positions read: [[0. 0. -10.]]
=== Starting algorithm loop ===
--- Iteration 1/100 ---
    Best fitness so far: 68.234
    x_best: [ 0.  0. -10.]
--- Iteration 2/100 ---
    Best fitness so far: 61.102
...

```

In Gazebo: the drone will start moving, changing direction each iteration as the algorithm computes new setpoints. In QGC: you will see the drone's position on the map updating continuously.

12.4 Confirm Success Criteria

- Best fitness decreases over iterations (not necessarily every single step, but trending down overall)
- Drone visibly moves in Gazebo toward the target region
- A log JSON file appears in the logs/ directory after completion
- Drone lands automatically when MAX_ITER iterations are complete

STEP 13: Add the Second Drone

Only do this after Step 12 works correctly for a single drone.

13.1 Launch the Second PX4 Instance

In Terminal 3 (a new WSL tab), find the correct airframe name first:

```
ls ~/robotics_sim/PX4-Autopilot/build/px4_sitl_default/rootfs/1/etc/init.d-
posix/airframes/ | grep -i x500
```

Then launch instance 1 using that exact name:

```
cd ~/robotics_sim/PX4-Autopilot
PX4_SIM_MODEL=gz_x500 PX4_GZ_WORLD=windy PX4_SIM_HOST_ADDR=127.0.0.1 \
PX4_INSTANCE=1 ./build/px4_sitl_default/bin/px4 -i 1
```

Verify both x500_0 and x500_1 appear in Gazebo's Entity Tree panel.

13.2 Update Settings for Two Drones

In config/settings.py, change:

```
N_AGENTS = 2
DRONE_PORTS = [14540, 14541]
```

13.3 Run the Two-Drone Algorithm

```
python3 main.py
```

You should now see both drones taking off, then moving according to the SMA+Stigmergy algorithm, with their movements subtly coordinated through the shared pheromone field even though they compute their own setpoints independently.

STEP 14: Verify the Jamming Zone Is Working

This is the key novelty of your algorithm. To confirm it works:

10. Set TARGET to a position on the far side of the JAMMED_ZONE in settings.py, so the drones must pass through the jammed region to reach the target.
11. Run the algorithm and watch the log output. When a drone's position is inside the jammed zone coordinates, it should still make progress (following pheromone) rather than freezing.
12. Temporarily set PHEROMONE_WEIGHT = 0.0 in settings.py and run again. With no stigmergy, jammed drones should behave worse (random-walking or stalling) since they have no pheromone fallback. This comparison is the empirical proof of your algorithm's contribution.
13. Restore PHEROMONE_WEIGHT = 0.2 and run again to confirm stigmergy restores performance.

TIP: Log the fitness values for both the stigmergy-enabled and stigmergy-disabled runs and compare them in a graph. This comparison plot is strong evidence for your paper.

STEP 15: Save and Protect Your Work

Now that everything is working, establish safe habits:

14. Never save Gazebo worlds over PX4 source files. Always use Save World As to ~/robotics_sim/my_worlds/
15. Check PX4 source files are unmodified before ending each session:

```
cd ~/robotics_sim/PX4-Autopilot/Tools/simulation/gz
git status --porcelain | grep -i x500
# Should return nothing. If it shows M, restore:
git checkout -- models/x500/model.sdf
```

16. Back up your algorithm scripts regularly:

```
cp -r ~/robotics_sim/swarm_project ~/robotics_sim/swarm_project_backup_$(date +%Y%m%d)
```

17. Save logs after every meaningful run before they are overwritten.

Summary: Complete Command Reference

Term	What It Means
Start simulation	PX4_SIM_HOST_ADDR=127.0.0.1 make px4_sitl gz_x500_windy
Run algorithm (1 drone)	cd ~/robotics_sim/swarm_project && python3 main.py
Kill everything	sudo pkill -9 -f px4 && sudo pkill -9 -f gz
Full WSL reset	wsl --shutdown (in PowerShell)
Check GCS connection	mavlink status (in pxh> shell)
Arm drone manually	commander arm (in pxh> shell)
Land drone manually	commander land (in pxh> shell)
Restore model file	cd ../gz && git checkout -- models/x500/model.sdf
Check algorithm imports	python3 -c 'from algorithm.sma_core import sma_step; print(OK)'
View logs	cat ~/robotics_sim/swarm_project/logs/run_*.json